obra/superpowers/**SKILL.md**

---

name:

# SUPER-POWERS

description: Skills que ensinam agentes de IA a desenvolver como engenheiros

author: **Regis Araujo Melo**

---

Agentes & Skills para desenvolvimento · skills.sh/obra



QUEM APRESENTA

# REGIS ARAUJO MELO

## Engenheiro iOS Sênior

Desenvolvedor iOS sênior com mais de 20 anos em tecnologia, com foco em apps nativos para os setores financeiro, varejo e mobilidade. Arquitetura modular, segurança, acessibilidade e performance, evolução de código legado e entrega contínua.

**LinkedIn**
linkedin.com/in/regisaraujomelo

ROTEIRO · 60 MINUTOS

# O QUE VAMOS FAZER

## CONCEITOS

**0:00 Abertura**
checagem da tarefa prévia no chat

**0:03 O problema e o agente**
por que chat isolado não basta

**0:07 Skills e o superpowers**
o que é uma skill e o mapa das 14

**0:12 Pensar antes de codar**
brainstorming em detalhe

**0:17 No Copilot CLI**
instalação e regras do laboratório

## PRÁTICA

**0:20 Laboratório · 25 min**
brainstorming, plano e primeira tarefa com TDD

**0:45 O resto do fluxo**
qualidade, escala, revisão e entrega

**0:52 Encerramento**
retrospectiva, cuidados e próximos passos

**0:57 Perguntas**

O PROBLEMA

# RÁPIDO NÃO É O MESMO QUE BOM

Um agente escreve código em segundos, mas:

- Começa a codar antes de entender o problema
- Não escreve testes, ou escreve depois só para “passar”
- Chuta a causa de um bug em vez de investigar
- Diz “pronto” sem ter verificado

## E se desse para ensinar processo ao agente?

CONCEITO

# CHAT ISOLADO VS AGENTE

| | Chat isolado | Agente |
|---|---|---|
| | você ⇄ chat<br>uma pergunta, uma resposta | planeja → age → observa ↻<br>repete até cumprir o objetivo |
| CONTEXTO | Só o que você cola na conversa. | **Lê o repositório: código, docs e histórico.** |
| AÇÕES | Responde com texto. | **Edita arquivos e roda comandos.** |
| VERIFICAÇÃO | Você testa depois, por conta própria. | **Roda os testes, lê o erro e corrige.** |
| CONTINUIDADE | Cada conversa começa do zero. | **Segue as instruções e as skills do projeto.** |
| SEU PAPEL | Copiar, colar e executar. | **Definir o objetivo e revisar o resultado.** |

**Skills entram nesse ciclo: dizem ao agente como trabalhar a cada volta.**

CONCEITO

# SKILL = PROCESSO EMPACOTADO

Uma pasta com um `SKILL.md`: instruções, checklists e exemplos que o agente **carrega sozinho quando a situação pede**.

```
---
name: brainstorming
description: Use antes de qualquer
  trabalho criativo…
---
```

**Prompt**
Você repete a cada conversa.

**Ferramenta**
Dá ao agente uma capacidade: ler arquivo, rodar comando.

**Skill**
Dá ao agente um método: reutilizável, versionado e compartilhável.

O MAPA DO SUPERPOWERS

# 14 SKILLS, 4 MOMENTOS DO TRABALHO

## 01 Pensar antes de codar

- brainstorming
- writing-plans
- executing-plans

## 02 Qualidade e disciplina

- test-driven-development
- systematic-debugging
- verification-before-completion

## 03 Escala com agentes

- subagent-driven-development
- dispatching-parallel-agents
- using-git-worktrees

## 04 Revisão e entrega

- requesting-code-review
- receiving-code-review
- finishing-a-development-branch
- writing-skills
- using-superpowers

uso no dia a dia | sugestão

# 01

# PENSAR ANTES DE CODAR

| SEM A SKILL | COM A SKILL, O AGENTE… |
| --- | --- |
| **`brainstorming`**<br>Pedido vago vira código errado. | faz perguntas uma a uma, propõe 2 ou 3 abordagens e só avança com um design aprovado. |
| **`writing-plans`**<br>Tarefa grande vira improviso. | quebra o trabalho em passos pequenos, com arquivos, testes e critério de pronto. |
| **`executing-plans`**<br>O plano é esquecido no meio do caminho. | executa o plano em lotes, com pontos de revisão. |

**Menos retrabalho: o código nasce do entendimento.**

PENSAR ANTES DE CODAR · EM DETALHE

# brainstorming

## QUANDO USAR

Antes de qualquer trabalho criativo: uma feature nova, um componente, uma mudança de comportamento.

## REGRA DE OURO

> **Nenhum código antes de um design aprovado. Vale até para o que "parece simples".**

## O QUE GERA

`docs/superpowers/specs/AAAA-MM-DD-tema-design.md`

**Na prática:** na história da lista de bancos, o agente pergunta como ordenar, se a busca é por nome ou código e o que mostrar com a lista vazia, antes de escrever código.

## O PROCESSO

1. **Explora o contexto:** arquivos, docs e commits recentes.
2. **Pergunta uma coisa por vez,** de preferência com múltipla escolha.
3. **Propõe 2 ou 3 abordagens** com prós, contras e uma recomendação.
4. **Apresenta o design em partes** e pede aprovação de cada uma.
5. **Escreve o spec e faz commit,** depois revisa o texto e pede a revisão de quem pediu.
6. **Passa a vez para** `writing-plans`.

NA PRÁTICA

# SUPERPOWERS NO COPILOT CLI

## Instalar

PADRÃO DO SKILLS.SH

```
npx skills add obra/superpowers -g -a github-copilot
```

OU COMO PLUGIN DO COPILOT CLI

```
copilot plugin marketplace add obra/superpowers-marketplace
copilot plugin install superpowers@superpowers-marketplace
```

## Conferir

Dentro do `copilot`, digite `/skills list`: devem aparecer `brainstorming`, `writing-plans` e as demais.

## Usar

Cite a skill com barra no pedido: `Use a skill /brainstorming para esta história…`

Skills avulsas: **`.github/skills/`** no repositório ou **`~/.copilot/skills/`** na sua máquina.

ANTES DE COMEÇAR

# REGRAS DO LABORATÓRIO

### Só branch local

`git switch -c workshop/seu-nome` e nada de push para o remoto.

### Nada sensível

Não cole segredos, tokens nem dados de clientes no agente.

### Checkpoints no chat

Ao fim de cada etapa, cole no chat do Teams o que o slide pedir.

### Copie do guia

Os comandos e pedidos estão no guia do participante, no chat. Não copie da tela.

Travou? Escreva no chat e acompanhe a tela de quem avançou.

LABORATÓRIO · ETAPA 1 DE 3

10 min

# BRAINSTORMING

1. Crie a branch: `git switch -c workshop/seu-nome`
2. Abra o agente na raiz do repositório: `copilot`
3. Cole o pedido da etapa 1 do guia (ele usa `/brainstorming` e traz a história).
4. **Responda às perguntas** e aprove o design.

A HISTÓRIA

**Como** cliente que quer trazer investimentos de outro banco, **quero** escolher o banco de origem numa lista, **para** iniciar a portabilidade sem digitar dados do banco.

{"code": "001", "institutionType": "1", "name": "Banco do Brasil", "cnpj": "00000000000191"}

**Checkpoint no chat:** a pergunta mais útil que o agente fez.

LABORATÓRIO · ETAPA 2 DE 3

5 min

# O PLANO

1. Na mesma sessão, cole o pedido da etapa 2 do guia (`/writing-plans`).
2. Leia o plano: **tarefas pequenas**, com arquivos e testes em cada uma.
3. Algo fora do recorte da sua stack? Peça para cortar.

**Checkpoint no chat:** quantas tarefas o seu plano tem.

LABORATÓRIO · ETAPA 3 DE 3

10 min

# PRIMEIRA TAREFA COM TDD

1. Cole o pedido da etapa 3 do guia (`/test-driven-development`, só a Tarefa 1).
2. Confira que o teste **falha primeiro**: vermelho.
3. Deixe o agente implementar e rodar de novo: **verde**.
4. Revise com `git diff`. Commit só local, sem push.

**Checkpoint no chat:** a saída do teste passando. Não terminou? O guia mostra como continuar.

# 02

# QUALIDADE E DISCIPLINA

| SEM A SKILL | COM A SKILL, O AGENTE… |
| --- | --- |
| **`test-driven-development`** | |
| Teste escrito depois só confirma o código. | segue vermelho, verde, refatorar: o teste falha primeiro. |
| **`systematic-debugging`** | |
| O bug é “consertado” no chute. | reproduz, isola e encontra a causa raiz antes de corrigir. |
| **`verification-before-completion`** | |
| “Pronto!” sem prova. | só declara pronto com evidência: teste rodado, saída conferida. |

Confiança: cada afirmação tem evidência.

# 03

# ESCALA COM AGENTES

Mais trabalho em paralelo sem perder o controle.

| SEM A SKILL | COM A SKILL, O AGENTE… |
| --- | --- |
| **`subagent-driven-development`** | |
| Um agente só se perde num contexto longo. | entrega cada tarefa do plano a um subagente novo, com revisão entre tarefas. |
| **`dispatching-parallel-agents`** | |
| Problemas independentes resolvidos em fila. | dispara vários agentes em paralelo, um por problema. |
| **`using-git-worktrees`** | |
| Tarefas paralelas se atropelam no mesmo diretório. | isola cada frente de trabalho na sua própria worktree. |

# 04

# REVISÃO E ENTREGA

| SEM A SKILL | COM A SKILL, O AGENTE… |
| --- | --- |
| **`requesting-code-review`**<br>O agente revisa o próprio trabalho, com o próprio viés. | pede uma revisão independente antes de integrar. |
| **`receiving-code-review`**<br>Aceita toda sugestão sem pensar. | avalia cada sugestão com rigor técnico e contesta quando necessário. |
| **`finishing-a-development-branch`**<br>Branch pronta e esquecida. | apresenta opções claras: merge, PR ou limpeza. |
| **`writing-skills`**<br>O time aprende algo e o conhecimento se perde. | transforma o aprendizado numa nova skill, testada. |
| **`using-superpowers`**<br>O agente não sabe que as skills existem. | procura e usa a skill certa antes de agir. É a porta de entrada. |

O ciclo fecha e o conhecimento vira skill.

JUNTANDO TUDO

# O FLUXO COMPLETO

01 brainstorming › 02 writing-plans › 03 using-git-worktrees › 04 subagent-driven-development + test-driven-development › 05 requesting-code-review › 06 verification-before-completion › 07 finishing-a-development-branch

COMO COMEÇAR

```
# padrão do skills.sh
$ npx skills add obra/superpowers -g -a github-copilot
# ou como plugin do Copilot CLI
$ copilot plugin marketplace add obra/superpowers-marketplace
$ copilot plugin install superpowers@superpowers-marketplace
```

Depois, dentro do **copilot**: **/skills list**

skills.sh/obra · github.com/obra/superpowers

**Agentes rápidos + método = software confiável.**

COM OS PÉS NO CHÃO

# CUIDADOS E LIMITES

### Mudança pequena

Correção de uma linha não pede brainstorming nem plano.

### Custa tempo e tokens

O processo completo compensa quando há dúvida de regra ou risco.

### Skills de terceiros

Revise o `SKILL.md` antes de instalar: ele dá instruções ao seu agente.

### Você continua responsável

Revise o diff e rode os testes antes de abrir o PR.

DEPOIS DO WORKSHOP

# PRÓXIMOS PASSOS

1. **Termine o laboratório** com a seção “Como continuar” do guia.
2. Use **/brainstorming** na sua próxima história real.
3. Conheça as **skills da sua plataforma**: iOS, Android e Java, no apêndice.
4. **Traga o resultado** na próxima reunião: o que funcionou e o que não.

## Perguntas?

APÊNDICE

# MATERIAL DE APOIO

Para usar se sobrar tempo ou surgir uma pergunta.

1. **obra e o skills.sh** — o autor e o catálogo
2. **writing-plans** e **executing-plans** em detalhe
3. **Skills por plataforma** — iOS, Android e backend Java

AUTOR E CATÁLOGO

# UM CATÁLOGO DE MÉTODOS

### obra

Jesse Vincent publica skills que codificam práticas de engenharia sênior para agentes.

### skills.sh

Diretório aberto de skills: você encontra, instala e usa no seu agente.

OUTROS PACOTES DO AUTOR

| | |
|---|---|
| **episodic-memory** | o agente lembra de conversas anteriores |
| **the-elements-of-style** | escrita clara e concisa |
| **superpowers-lab** | experimentos: tmux, funções duplicadas… |
| **superpowers-chrome** | navegar na web pelo Chrome |

O principal é o **superpowers** →

PENSAR ANTES DE CODAR · EM DETALHE

# writing-plans

## QUANDO USAR

Com um spec aprovado em mãos, antes de tocar no código.

## REGRA DE OURO

> **Escrever para um dev que não conhece o projeto: nada de “TODO” ou “implementar depois”.**

## O QUE GERA

`docs/superpowers/plans/AAAA-MM-DD-feature.md`

**Na prática:** a lista de bancos vira tarefas como “modelo `Institution` + teste”, “use case com busca + teste” e “estados do ViewModel + teste”.

## O PROCESSO

1. **Mapeia os arquivos** e a responsabilidade de cada um.
2. **Quebra em tarefas pequenas:** cada passo leva de 2 a 5 minutos.
3. **Em cada tarefa:** teste que falha, implementação, teste passando, commit.
4. **Escreve tudo por extenso:** caminhos exatos, código completo, comando e saída esperada.
5. **Revisa o plano contra o spec:** cobertura, pendências e nomes consistentes.
6. **Oferece a execução:** com subagentes ou na própria sessão.

PENSAR ANTES DE CODAR · EM DETALHE

# executing-plans

## QUANDO USAR

Quando há um plano escrito para executar com pontos de revisão. Com subagentes disponíveis, prefira `subagent-driven-development`.

## REGRA DE OURO

> **Travou? Para e pergunta. Não adivinha e não força o caminho.**

## O QUE GERA

Código commitado num branch, pronto para integrar.

**Na prática:** cada tarefa da lista de bancos roda na branch `workshop/seu-nome`; se o teste do use case falhar sem motivo claro, o agente para e pergunta.

## O PROCESSO

1. **Prepara um espaço isolado** com `using-git-worktrees`.
2. **Lê o plano com olhar crítico** e levanta dúvidas antes de começar.
3. **Executa tarefa por tarefa:** segue os passos à risca e roda as verificações.
4. **Para no primeiro bloqueio:** dependência faltando, teste falhando, instrução confusa.
5. **Fecha com `finishing-a-development-branch`**: merge, PR ou limpeza.

SKILLS POR PLATAFORMA · 1 DE 3

# iOS · SWIFT

### `swiftui-pro`
`twostraws/swiftui-agent-skill`

Revisa código SwiftUI: APIs modernas, manutenção e performance.

### `swiftui-expert-skill`
`avdlee/swiftui-agent-skill`

Escreve e refatora SwiftUI: estado com `@Observable`, composição de views e listas.

### `swift-concurrency`
`avdlee/swift-concurrency-agent-skill`

Diagnostica concorrência, troca callbacks por async/await e guia a migração para o Swift 6.

### `swift-testing-pro`
`twostraws/swift-testing-agent-skill`

Escreve e revisa testes com o framework Swift Testing.

### `ios-accessibility`
`dpearson2699/swift-ios-skills`

Constrói e audita acessibilidade: VoiceOver, Voice Control e Dynamic Type.

**COMBINA COM O SUPERPOWERS**

`swift-testing-pro` + `test-driven-development`: o teste vem primeiro, já com as APIs do Swift Testing.

No Copilot CLI: copie a pasta da skill para **`.github/skills/`** ou **`~/.copilot/skills/`**

Revise o **`SKILL.md`** antes de instalar.

SKILLS POR PLATAFORMA · 2 DE 3

# ANDROID · KOTLIN

### migrate-xml-views-to-jetpack-compose

android/skills oficial

Roteiro guiado para migrar telas em XML para Jetpack Compose.

### navigation-3

android/skills oficial

Instala e migra para o Navigation 3: deep links, múltiplas pilhas e diálogos.

### edge-to-edge

android/skills oficial

Adapta apps Compose para a tela de borda a borda e corrige os problemas comuns.

### kotlin-tooling-java-to-kotlin

kotlin/kotlin-agent-skills JetBrains

Converte código Java em Kotlin idiomático.

### kotlin-coroutines-flows

affaan-m/ecc

Coroutines e Flow: concorrência estruturada, StateFlow, erros e testes.

**COMBINA COM O SUPERPOWERS**

`migrate-xml-views-to-jetpack-compose` + `writing-plans`: a migração vira um plano tela a tela.

No Copilot CLI: copie a pasta da skill para **`.github/skills/`** ou **`~/.copilot/skills/`**

Revise o **`SKILL.md`** antes de instalar.

SKILLS POR PLATAFORMA · 3 DE 3

# BACKEND · JAVA

### java-springboot

github/awesome-copilot GitHub

Boas práticas para aplicações Spring Boot.

### create-spring-boot-java-project

github/awesome-copilot GitHub

Gera o esqueleto de um projeto Spring Boot em Java.

### java-junit

github/awesome-copilot GitHub

Testes com JUnit 5, incluindo testes orientados a dados.

### springboot-security

affaan-m/ecc

Spring Security: autenticação, autorização, validação, CSRF, segredos e rate limiting.

### springboot-tdd

affaan-m/ecc

TDD com JUnit 5, Mockito, MockMvc, Testcontainers e JaCoCo.

COMBINA COM O SUPERPOWERS

`springboot-security` + `requesting-code-review`: segurança revisada por um olhar independente antes do merge.

No Copilot CLI: copie a pasta da skill para **`.github/skills/`** ou **`~/.copilot/skills/`**

Revise o **`SKILL.md`** antes de instalar.